Sandboxing a coding agent

Bounding what it can read, write, and reach

Stefano Schuppli et al.

Stock container primitives — rootless Podman, an unprivileged user, a read-only root, no route out except one allowlisted proxy. Pointed at CSCS inference, not a vendor endpoint.

What we are going to set up

Two container images and one shell command. After that, a coding agent that can only touch the project you started it in.

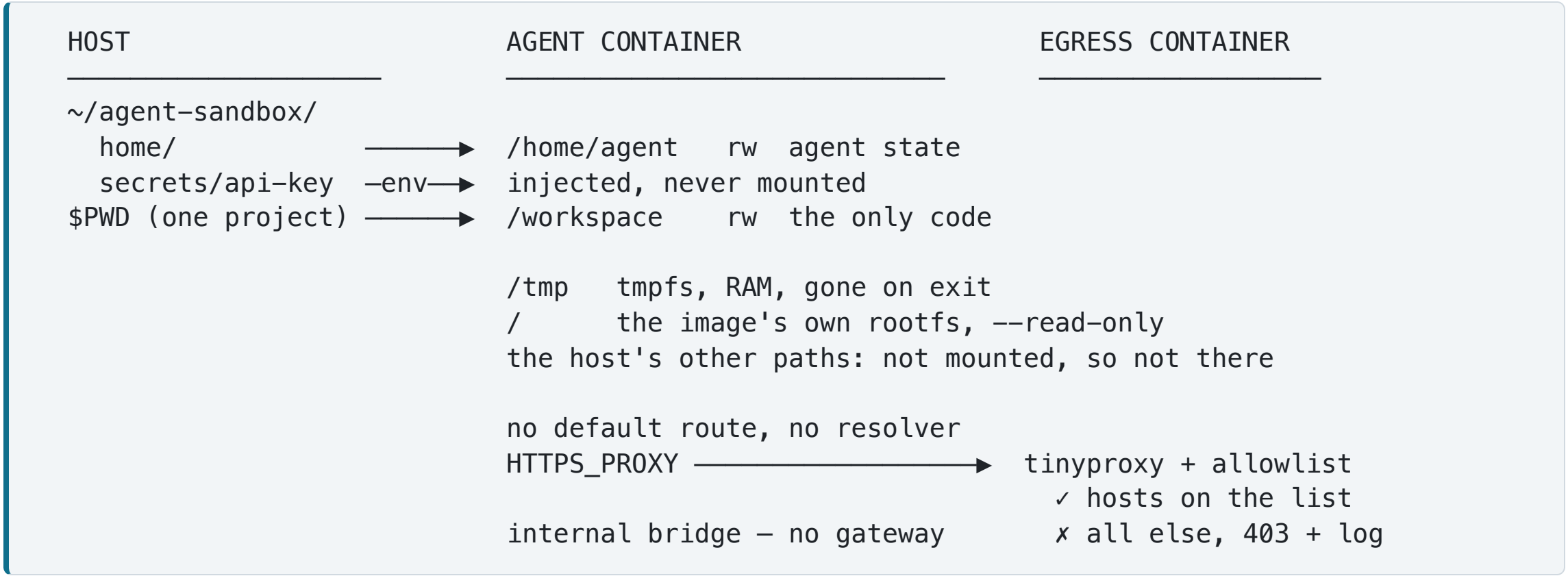
Type `oc` in any project and land in an OpenCode session that:

- runs as an **unprivileged user**, not root
- has a **home of its own** — `~/agent-sandbox/home`: no SSH keys, no cloud creds. One home shared by every session, so history and caches persist
- has a **`/tmp` of its own** — RAM-backed, gone on exit
- has a **read-only root filesystem**
- takes **one project as its workspace** — the directory you started it in; nothing else of yours is mounted
- reaches only what an **allowlist you control** permits — there is no other route off its network

Small enough that you can stop reading every command before you approve it.

What the box looks like

Two containers: the agent, and the only thing it is allowed to talk to.



There is no default route in that container at all, and the proxy does the name resolution.

Building it

Two one-off steps: get a key, build two images.

```
# 1. a key from https://ui.inference.cscs.ch/login  (shown once — set a budget)
umask 077
printf '%s' 'your-key' > ~/agent-sandbox/secrets/cscs-api-key

# 2. build — the agent, and the only thing it is allowed to talk to
podman build -t opencode-cscs:1.18.17 .
podman build -f Dockerfile.proxy -t agent-egress-proxy:1 .
```

> The `apk add` list is a capability list — everything on it, the model can run. `openssh-client` and `python3` are on it on purpose: an agent that cannot ssh to a node or run a script gets used *outside* the sandbox instead.
>
> **Reach is bounded by the proxy, not by the package list.**

The `oc` command

Read it first as an alias: one word standing in for a long `podman run`.

```
# the shape of it
alias oc='podman run --rm -it  …about twenty flags…  opencode-cscs:1.18.17'
```

It ships as a shell **function**, not an alias, because three things must happen first:

- read the key from a file and pass it in the **environment**, never in an argument list where `ps` would show it
- **refuse to start** if you are sitting in your `$HOME` — that would hand over everything
- bring the egress proxy up if it is not already running

```
oc                                      # interactive TUI
oc run "add a test for the retry path"  # one-shot, no TUI
AGENT_EGRESS=none oc                    # same, but with no network at all
oc-proxy-log                            # live: everywhere it tried to go
oc-doctor                               # is everything in place?
```

The allowlist — which hosts it can reach

One file. Every line is a hostname the proxy will forward to; anything else gets a 403 and a log entry.

```
# egress-allowlist — one pattern per line, matched against the WHOLE hostname
api.inference.cscs.ch      # the inference endpoint — without it, nothing works
models.opencode.ai         # model catalogue, fetched at startup
#github.com                # uncomment to allow clone and fetch — and exfiltration
```

What it stops

- posting the workspace to a pastebin
- `curl … | sh` from a host named in a README
- the cloud metadata endpoint
- scanning your laptop and the office LAN
- `pip install` — **which you will feel.** Not a free win

What it does not

- **exfiltration through an allowed host.** Your whole context already goes to CSCS by design
- **TLS inspection.** It matches the hostname in the `CONNECT`, nothing more
- the bridge gateway *is* the host — check what listens there
- Docker: its embedded resolver gets in the way. Verified on Podman only

Gotchas — every one found by running it

- **Matching is whole-hostname.** `theguardian.com` does not cover `www.theguardian.com`; you need `*.theguardian.com` as well. The proxy log names the exact host it refused.
- **The `--tmpfs` defaults are not what the docs led us to expect.** Docker documents `rw,noexec,nosuid,size=65536k`; Podman 5.6.2 gives `rw,nosuid,nodev` — exec **allowed**, size **unbounded**. So `exec` is an override on one engine and a no-op on the other; `size=1g` is a cap on one and a raise on the other.
- **The provider SDK does not need npm.** The config says `"npm": "@ai-sdk/anthropic"` and opencode does ask — but every `@ai-sdk/*` is compiled into the binary. Blocked at the proxy, it falls back and works.
- **Podman builds OCI by default, where `SHELL` is ignored** — silently dropping `set -e`. Use explicit `set -eux` in each `RUN`.
- **An internal network kills the proxy's own DNS too** — create it `--disable-dns` so the proxy inherits the host's resolvers instead.

Take it apart YOUR TURN

Suggested experiments, and what each one demonstrates.

- `AGENT_EGRESS` picks the network mode: `proxy` (default), `open`, `none`.
- Run it with `none`: the TUI starts and the file tools still work, but no reply arrives. Inference runs at CSCS, not in the container.
- Add `github.com` to the allowlist, then look at what else that opens — a repo it can push to is a route your source can leave by.
- Delete a line from the `apk add` and rebuild; check what busybox still provides in its place.
- Flip `bash` from `allow` to `ask` and work for an hour. That is the trade the shipped default is making.

Future work

- no TLS inspection — a MITM proxy is a much bigger commitment
- one shared proxy and one allowlist per machine, not per project
- the egress step is Podman-only as written
- the workspace is still writable — **commit before you start**